



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

“Práctica #1: Hilos (Equipo)”

Sistemas Distribuidos y Paralelos

M.I.I. Carlos Adrián Pérez Cortez

Grupo: 004

Hora: N5

Salón: 4207

Equipo 9

MATRICULA	NOMBRE	PE
1952947	Heidi Pamela Martínez Martínez	ITS
1845517	Andrea Monserrat Méndez Flores	ITS
2109147	Alexia Hernandez Andrade	ITS
2004428	Alejandro Sauqué Pérez	ITS
2077884	Melida Krystal Mendoza Ventura	ITS

Ciudad Universitaria, San Nicolás de los Garza, Nuevo León a 10 de
septiembre de 2024

CAPTURA DE PANTALLA DEL CÓDIGO EN JAVA

```
1  import java.io.BufferedReader;
2  import java.io.FileReader;
3  import java.io.IOException;
4  import java.util.List;
5  import java.util.concurrent.Callable;
6  import java.util.concurrent.ExecutorService;
7  import java.util.concurrent.Executors;
8  import java.util.concurrent.Future;
9  import java.util.ArrayList;
10 import java.util.Scanner;
11
12 public class ThreadedAverageCalculator {
13     Run | Debug
14     public static void main(String[] args) throws Exception {
15         String fileName = "aleatorios.txt";
16         List<Integer> numbers = new ArrayList<>();
17
18         // Leer número máximo de particiones del usuario
19         Scanner scanner = new Scanner(System.in);
20         System.out.print(s:"Ingrese el numero maximo de particiones (hilos): ");
21         int maxPartitions = scanner.nextInt();
22         scanner.close();
23
24         // Lee los números aleatorios del archivo y los almacena en numbers
25         try (BufferedReader br = new BufferedReader(new FileReader(fileName))) {
26             String line = br.readLine();
27             String[] numberStrings = line.split(regex:",");
28             for (String numberString : numberStrings) {
29                 numbers.add(Integer.parseInt(numberString));
30             }
31         } catch (IOException e) {
32             e.printStackTrace();
33         }
34
35         int totalNumbers = numbers.size();
36
37         // Ejecutar el cálculo con particiones desde 1 hasta el número máximo
38         for (int partitionCount = 1; partitionCount <= maxPartitions; partitionCount++) {
39             System.out.println("\n=== Ejecutando con " + partitionCount + " particion(es) ===");
40             calculateAverageWithPartitions(numbers, partitionCount, totalNumbers);
41         }
42
43         private static void calculateAverageWithPartitions(List<Integer> numbers, int partitionCount, int totalNumbers) throws Exception {
44             int partitionSize = totalNumbers / partitionCount;
45             ExecutorService executor = Executors.newFixedThreadPool(partitionCount);
46             List<Future<Double>> results = new ArrayList<>();
47
48             long startTime = System.currentTimeMillis();
49
50             // Asignar segmentos de la lista a los hilos
51             for (int i = 0; i < partitionCount; i++) {
52                 int start = i * partitionSize;
53                 int end = (i == partitionCount - 1) ? totalNumbers : (i + 1) * partitionSize;
54                 List<Integer> segment = numbers.subList(start, end);
55                 Callable<Double> calculator = new AverageCalculator(segment);
56                 Future<Double> result = executor.submit(calculator);
57                 results.add(result);
58             }
59         }
60     }
61 }
```

```

59
60     try {
61         // Obtener los resultados de los hilos y calcular el promedio total
62         double totalSum = 0;
63
64         for (Future<Double> result : results) {
65             totalSum += result.get() * partitionSize;
66         }
67
68         double overallAverage = totalSum / totalNumbers;
69         System.out.println("El promedio total es: " + overallAverage);
70     } catch (Exception e) {
71         e.printStackTrace();
72     } finally {
73         executor.shutdown();
74     }
75
76     long endTime = System.currentTimeMillis();
77     long executionTime = endTime - startTime;
78     System.out.println("Tiempo de ejecucion: " + executionTime + " milisegundos");
79 }
80
81
82 class AverageCalculator implements Callable<Double> {
83     private List<Integer> numbersToCompute;
84
85     public AverageCalculator(List<Integer> numbersToCompute) {
86         this.numbersToCompute = numbersToCompute;
87     }
88
89     @Override
90     public Double call() {
91         double sum = 0;
92         for (int num : numbersToCompute) {
93             sum += num;
94         }
95         return sum / numbersToCompute.size();
96     }
97 }

```

RESULTADO

```
(base) C:\Users\heidi\Desktop>java ThreadedAverageCalculator
Ingrese el numero maximo de particiones (hilos): 12
```

```
=== Ejecutando con 1 particion(es) ===
El promedio total es: 499.549762
Tiempo de ejecucion: 20 milisegundos
```

```
=== Ejecutando con 2 particion(es) ===
El promedio total es: 499.549762
Tiempo de ejecucion: 14 milisegundos
```

```
=== Ejecutando con 3 particion(es) ===
El promedio total es: 499.5492628616313
Tiempo de ejecucion: 2 milisegundos
```

```
=== Ejecutando con 4 particion(es) ===
El promedio total es: 499.549762
Tiempo de ejecucion: 3 milisegundos
```

```
=== Ejecutando con 5 particion(es) ===
El promedio total es: 499.549762
Tiempo de ejecucion: 1 milisegundos
```

```
=== Ejecutando con 6 particion(es) ===
El promedio total es: 499.5477668057119
Tiempo de ejecucion: 1 milisegundos
```

```
=== Ejecutando con 7 particion(es) ===
El promedio total es: 499.5492630584707
Tiempo de ejecucion: 1 milisegundos
```

```
=== Ejecutando con 8 particion(es) ===
El promedio total es: 499.549762
Tiempo de ejecucion: 1 milisegundos
```

```
=== Ejecutando con 9 particion(es) ===
El promedio total es: 499.549263124217
Tiempo de ejecucion: 1 milisegundos
```

```
=== Ejecutando con 10 particion(es) ===
El promedio total es: 499.549762
Tiempo de ejecucion: 1 milisegundos
```

```
=== Ejecutando con 11 particion(es) ===
El promedio total es: 499.5492629810472
Tiempo de ejecucion: 13 milisegundos
```

```
=== Ejecutando con 12 particion(es) ===
El promedio total es: 499.5477669686934
Tiempo de ejecucion: 1 milisegundos
```

Ejecutamos el programa varias veces y observamos que el rendimiento varía en cada ejecución. En promedio, utilizar 6 hilos resulta ser la opción más eficiente y rápida. Sin embargo, en algunas ocasiones, el rendimiento con 8 hilos también ha mostrado mejoras significativas, a veces incluso alcanzando tiempos de ejecución de 0 milisegundos (menos de 1 milisegundo). Por lo tanto, aunque el número óptimo de hilos generalmente parece ser 6, es importante considerar que en ciertos casos 8 hilos puede ofrecer un rendimiento superior. En la mayoría de las ejecuciones, 6 hilos siguen siendo la opción preferida.

Conclusiones

1952947 - Heidi Pamela Martínez Martínez

Conclusión

Heidi Pamela Martínez Martínez 1952947

Los hilos permiten la ejecución simultánea de múltiples tareas dentro de un mismo proceso, lo que puede mejorar significativamente el rendimiento en tareas que involucren grandes volúmenes de datos. En esta práctica, usando como base las prácticas anteriores, en donde incluimos al usuario en el proceso definimos un número de hilos definidos por el mismo de manera secuencial.

El uso de múltiples hilos se aplica para dividir la carga de trabajo y acelerar el procesamiento, claro, la elección de número de hilos óptimo depende también del número de núcleos lógicos del procesador.

En este caso, 6 hilos ofreció un equilibrio entre eficiencia y sobrecarga de gestión, haciéndolo ideal ya que mientras se probaba, algunas corridas reaccionaban mejor con 8 hilos, lo cual también funciona, pero en la mayoría fueron 6 hilos. Número óptimo = 6 - 8 hilos

Andrea Monserrat Méndez Flores

Conclusion:

En conclusion, para esta practica tratamos de utilizar el mayor numero de hilos posible, esperando así el tiempo de ejecución. Al ejecutar el programa con hilos en el proceso hace que este llegue a tener un tiempo de ejecución más rápido que uno normal o que ejecute de manera "lineal", pero esto igual depende de que tan variable sea el proceso que se requiere ejecutar.

Para este programa en especifico se descubrió que el número optimo de hilos era de 6.

Alexia Hernandez Andrade 2109147

Conclusión Tarea Practica #1 Hilos (equipo)

En la elaboración de esta practica, opino que pusimos a prueba nuestros conocimientos previos.

El enfoque de la actividad nos permitio evaluar como el uso de multiples hilos afecta el rendimiento del programa en comparación con la ejecución secuencial.

A medida que se aumentaba el numero de hilos veiamos una mejora pero tambien se destaco que apartir de cierto numero de hilos la mejora se vuelve negativa.

En conclusión, el numero de hilos puede mejorar la eficiencia, pero la sobrecarga de paralelización debe ser cuidadosamente gestionada para evitar perdidas de rendimiento.

Alejandro Sauqué Pérez

Conclusión Sistemas Distribuidos y Paralela

En esta práctica profundizamos más en el uso de hilos, específicamente en la comparación del rendimiento usando diferentes números de hilos o particiones. Me parece que en este ejercicio se nos está invitando a investigar cuál es el número ideal o mejor de hilos para esta tarea. Esto es interesante ya que pensaríamos que entre más mejor, pero llega un punto en el que si nos excedemos de hilos el programa se hace más lento. Me parece muy útil este tipo de prácticas, sin embargo creo que sería buena ver más acerca de los resultados que obtenemos porque con cada corrida salen ligeramente distintos. Entonces me gustaría saber porqué?

Melida Krystal Mendoza Ventura 2077884

El ejecutar un programa con hilos en el proceso hace que este llegue a tener un tiempo de ejecución más rápido que uno normal o que ejecute de manera "lineal", pero esto igual depende de que tan variable sea el proceso que se requiere ejecutar.

Dependiendo de la tarea, en la mayoría de los casos entre más hilos tengamos en el cuerpo del programa, siempre y cuando sean productivos/necesarios, menor será el tiempo que le tomará a este ejecutarse, no por tener demasiados hilos podremos obtener un resultado muy rápido, pues depende de que no tengamos hilos de más, ni estemos abarcando demasiado en uno solo.

La cantidad de tiempo es variable, en la mayoría de veces proporcionalmente mejor (más rápida).